

## ASSESSMENT TOOL - 2

### PSEUDOCODE WRITING

NAME : PRIYADHARSHINI K

REG NO : 192421236

COURSE : CSA0927 – PROGRAMMING IN JAVA

#### Q1 Student Management System [BL3 – Apply]

Write pseudocode to design a Student class with attributes (name, rollNo, marks) and methods to:

- Accept student details
- Calculate average marks
- Display result (Pass/Fail)

Apply encapsulation and data hiding by restricting direct access to attributes.

OOP Concepts: Encapsulation | Data Hiding | Classes & Objects

#### Your Answer

```
START
CLASS Student
  PRIVATE
    name
    rollNo
    marks1, marks2, marks3
    average

  PUBLIC
  METHOD acceptDetails()
    INPUT name
    INPUT rollNo
    INPUT marks1, marks2, marks3
  END METHOD

  METHOD calculateAverage()
    average ← (marks1 + marks2 + marks3) / 3
  END METHOD

  METHOD displayResult()
    PRINT "Student Name : ", name
    PRINT "Roll Number : ", rollNo
    PRINT "Average Marks: ", average
```

```

        IF average >= 50 THEN
            PRINT "Result : PASS"
        ELSE
            PRINT "Result : FAIL"
        END IF
    END METHOD
END CLASS

MAIN PROGRAM
    CREATE object student OF Student
    CALL student.acceptDetails()
    CALL student.calculateAverage()
    CALL student.displayResult()

STOP

```

## Q2 Library Book Management [BL3 – Apply]

Design a Book class with private attributes (title, author, ISBN, isAvailable). Implement methods to:

- Issue a book (change availability status)
- Return a book
- Display book details using getter methods

Demonstrate object creation for at least 3 books and simulate issue/return operations.

OOP Concepts: Encapsulation | Getters & Setters | Objects

### Your Answer

```

START
CLASS Book

PRIVATE
    title, author, ISBN
    isAvailable

PUBLIC
METHOD setBookDetails(t, a, i)
    title ← t
    author ← a
    ISBN ← i
    isAvailable ← TRUE
END METHOD

METHOD issueBook()
    IF isAvailable = TRUE THEN
        isAvailable ← FALSE
    ELSE
        DISPLAY "Book Already Issued"
    END IF
END METHOD

```

```
METHOD displayBookDetails()
    DISPLAY getTitle()
    DISPLAY getAuthor()
    DISPLAY getISBN()
    DISPLAY getAvailability()
END METHOD
```

```
METHOD getTitle()
    RETURN title
END METHOD
```

```
METHOD getAuthor()
    RETURN author
END METHOD
```

```
METHOD getISBN()
    RETURN ISBN
END METHOD
```

```
METHOD getAvailability()
    RETURN isAvailable
END METHOD
END CLASS
```

```
MAIN
CREATE book1, book2, book3 AS Book
```

```
CALL book1.setBookDetails("Java", "James", "101")
CALL book2.setBookDetails("Python", "Guido", "102")
CALL book3.setBookDetails("C++", "Bjarne", "103")
```

```
CALL book1.issueBook()
CALL book1.returnBook()
```

```
CALL book1.displayBookDetails()
CALL book2.displayBookDetails()
CALL book3.displayBookDetails()
```

```
STOP
```

---

### Q3 Employee Salary Calculation [BL3 – Apply]

10 marks

Write pseudocode for an Employee class with encapsulated attributes (empId, name, basicPay, designation). Implement methods to: •

Calculate gross salary (basic + HRA + DA)

- Calculate net salary after deductions (PF, tax)
- Display pay slip

Apply data hiding so salary components are not directly accessible.

OOP Concepts: Encapsulation | Data Hiding | Methods

### Your Answer

```
START
CLASS Employee
PRIVATE
    empId, name, designation
    basicPay
    HRA, DA
    PF, tax
    grossSalary, netSalary

PUBLIC
METHOD acceptDetails()
    INPUT empId
    INPUT name
    INPUT designation
    INPUT basicPay
END METHOD

METHOD calculateGrossSalary()
    HRA ← basicPay * 20 / 100
    DA ← basicPay * 10 / 100
    grossSalary ← basicPay + HRA + DA
END METHOD
```

```

METHOD calculateNetSalary()
  PF ← basicPay * 12 / 100
  tax ← basicPay * 5 / 100
  netSalary ← grossSalary - (PF + tax)
END METHOD

METHOD displayPaySlip()
  DISPLAY "Employee ID : ", empId
  DISPLAY "Name : ", name
  DISPLAY "Designation : ", designation
  DISPLAY "Basic Pay : ", basicPay
  DISPLAY "Gross Salary : ", grossSalary
  DISPLAY "Net Salary : ", netSalary
END METHOD
END CLASS

MAIN
CREATE emp AS Employee
CALL emp.acceptDetails()
CALL emp.calculateGrossSalary()
CALL emp.calculateNetSalary()
CALL emp.displayPaySlip()
STOP

```

#### Q4 Shape Area Calculator using Polymorphism [BL3 – Apply]

Create a Shape base class with an overridable calculateArea() method. Apply this to:

- Circle — area using radius
- Rectangle — area using length and breadth
- Triangle — area using base and height

Use method overriding to demonstrate runtime polymorphism. Call calculateArea() for each object.

OOP Concepts: Inheritance | Polymorphism | Method Overriding

#### Your Answer

```

START
CLASS Shape
  METHOD calculateArea()
    DISPLAY "Area Calculation"
  END METHOD
END CLASS

```

```
CLASS Circle EXTENDS Shape
    radius
    METHOD calculateArea()
        area  $\leftarrow 3.14 \times \text{radius} \times \text{radius}$ 
        DISPLAY "Area of Circle = ", area
    END METHOD
END CLASS
```

```
CLASS Rectangle EXTENDS Shape
    length, breadth
    METHOD calculateArea()
        area  $\leftarrow \text{length} \times \text{breadth}$ 
        DISPLAY "Area of Rectangle = ", area
    END METHOD
END CLASS
```

---

```
CLASS Triangle EXTENDS Shape
    base, height
    METHOD calculateArea()
        area  $\leftarrow (\text{base} \times \text{height}) / 2$ 
        DISPLAY "Area of Triangle = ", area
    END METHOD
END CLASS
```

```
MAIN
CREATE Circle c
SET c.radius  $\leftarrow 5$ 
```

```
CREATE Rectangle r
SET r.length  $\leftarrow 10$ 
SET r.breadth  $\leftarrow 4$ 
CREATE Triangle t
```

```
SET t.base  $\leftarrow 8$ 
SET t.height  $\leftarrow 6$ 
```

```
CALL c.calculateArea()
```

```
CLASS Triangle EXTENDS Shape
    base, height
    METHOD calculateArea()
        area ← (base × height) / 2
        DISPLAY "Area of Triangle = ", area
    END METHOD
END CLASS
```

MAIN

CREATE Circle c

SET c.radius ← 5

CREATE Rectangle r

SET r.length ← 10

SET r.breadth ← 4

CREATE Triangle t

SET t.base ← 8

SET t.height ← 6

CALL c.calculateArea()

CALL r.calculateArea()

CALL t.calculateArea()

STOP

## Q5 5. Vehicle Registration System [BL3 – Apply]

Design a Vehicle class with attributes (regNo, owner, fuelType). Extend it into:

- TwoWheeler — with engine capacity
- FourWheeler — with seating capacity and AC status

Apply inheritance to reuse common attributes and override a displayDetails() method in each subclass.

OOP Concepts: Inheritance | Method Overriding | Subclass

## Your Answer

```
START
CLASS Vehicle
  regNo, owner, fuelType
  METHOD setDetails(r, o, f)
    regNo ← r
    owner ← o
    fuelType ← f
  END METHOD

  METHOD displayDetails()
    DISPLAY "Registration No : ", regNo
    DISPLAY "Owner Name : ", owner
    DISPLAY "Fuel Type : ", fuelType
  END METHOD
END CLASS

CLASS TwoWheeler EXTENDS Vehicle
  engineCapacity
  METHOD displayDetails()
    CALL Vehicle.displayDetails()
    DISPLAY "Engine Capacity : ", engineCapacity
  END METHOD
END CLASS

CLASS FourWheeler EXTENDS Vehicle
  seatingCapacity
  ACstatus
  METHOD displayDetails()
    CALL Vehicle.displayDetails()
    DISPLAY "Seating Capacity : ", seatingCapacity
    DISPLAY "AC Status : ", ACstatus
  END METHOD
END CLASS

MAIN
CREATE TwoWheeler t
CALL t.setDetails("TN01AB1234", "Rahul", "Petrol")
SET t.engineCapacity ← 150

CREATE FourWheeler f
CALL f.setDetails("TN10CD5678", "Priya", "Diesel")
```



```
SET f.seatingCapacity ← 5
SET f.ACstatus ← "Yes"
```

```
CALL t.displayDetails()
CALL f.displayDetails()
STOP
```

#### Q6 6. Bank Account Hierarchy [BL4 – Analyze]

Develop pseudocode for a BankAccount superclass and two subclasses — SavingsAccount (with interest calculation) and CurrentAccount (with overdraft facility). Analyze and implement:

- Inheritance of common attributes (accNo, balance, holder)
- Method overriding for withdrawal behavior
- How overdraft protection differs from savings limit

Compare the behavior of withdraw() across both subclasses and justify the design choices.

OOP Concepts: Inheritance | Method Overriding | Polymorphism

#### Your Answer

```
START
CLASS BankAccount
  accNo, holder, balance

  METHOD setDetails(a, h, b)
    accNo ← a
    holder ← h
    balance ← b
  END METHOD

  METHOD withdraw(amount)
    DISPLAY "Withdraw Amount"
  END METHOD
END CLASS

CLASS SavingsAccount EXTENDS BankAccount
  interestRate

  METHOD calculateInterest()
    interest ← balance × interestRate / 100
    DISPLAY interest
  END METHOD
```

```

METHOD withdraw(amount)
  IF amount <= balance THEN
    balance ← balance - amount
    DISPLAY "Withdrawal Successful"
  ELSE
    DISPLAY "Insufficient Balance"
  END IF
END METHOD
END CLASS

CLASS CurrentAccount EXTENDS BankAccount
  overdraftLimit

  METHOD withdraw(amount)
    IF amount <= balance + overdraftLimit THEN
      balance ← balance - amount
      DISPLAY "Withdrawal Successful"
    ELSE
      DISPLAY "Overdraft Limit Exceeded"
    END IF
  END METHOD
END CLASS

```

```

MAIN
CREATE SavingsAccount s
CALL s.setDetails("101", "Priya", 10000)
CALL s.calculateInterest()
CALL s.withdraw(12000)

CREATE CurrentAccount c
CALL c.setDetails("102", "Rahul", 10000)
SET c.overdraftLimit ← 5000
CALL c.withdraw(12000)
STOP

```

## Q7 7. Hospital Patient Record System [BL4 – Analyze]

10 marks

Design a Patient base class and analyze how it extends into:

- InPatient — with ward number, admission date, and daily charges
- OutPatient — with appointment date and consultation fee

Analyze encapsulation requirements for sensitive data (diagnosis, prescription). Override a generateBill() method for each type and compare the billing logic differences. OOP Concepts: Encapsulation | Inheritance | Polymorphism

## Your Answer

```
START
CLASS Patient

    PRIVATE
        patientId, name
        diagnosis, prescription

    METHOD setDetails(id, n, d, p)
        patientId ← id
        name ← n
        diagnosis ← d
        prescription ← p
    END METHOD

    METHOD generateBill()
        DISPLAY "Generate Bill"
    END METHOD
END CLASS

CLASS InPatient EXTENDS Patient
    wardNo, admissionDate, dailyCharges, days
```

```
        METHOD generateBill()
            bill ← dailyCharges × days
            DISPLAY "InPatient Bill = ", bill
        END METHOD
    END CLASS
```

```
CLASS OutPatient EXTENDS Patient
    appointmentDate, consultationFee
```

```
        METHOD generateBill()
            bill ← consultationFee
            DISPLAY "OutPatient Bill = ", bill
        END METHOD
    END CLASS
```

```
MAIN
CREATE InPatient ip
CALL ip.generateBill()
```

```
CREATE OutPatient op
CALL op.generateBill()
```

```
STOP
```

**Q8 8. E-Commerce Product Catalog [BL4 – Analyze]**

10 marks

Analyze and design a Product class hierarchy for an e-commerce system:

- Electronics — with warranty, brand, voltage
- Clothing — with size, fabric, gender
- Grocery — with expiryDate and weight

Override an applyDiscount() method for each category with different discount rules. Analyze why polymorphism makes the cart checkout process flexible and extensible. OOP Concepts: Polymorphism | Inheritance | Method Overriding

**Your Answer**

START

CLASS Product

```
productId
productName
price
```

```
METHOD applyDiscount()
    DISPLAY "Discount Applied"
END METHOD
```

END CLASS

CLASS Electronics EXTENDS Product

```
warranty
brand
voltage
```

```
METHOD applyDiscount()
    discountPrice ← price - (price × 10 / 100)
    DISPLAY "Electronics Discount Price : ", discountPrice
END METHOD
```

END CLASS

CLASS Clothing EXTENDS Product

size

fabric

gender

METHOD applyDiscount()

discountPrice  $\leftarrow$  price - (price  $\times$  20 / 100)

DISPLAY "Clothing Discount Price : ", discountPrice

END METHOD

END CLASS

CLASS Grocery EXTENDS Product

expiryDate

weight

METHOD applyDiscount()

discountPrice  $\leftarrow$  price - (price  $\times$  5 / 100)

DISPLAY "Grocery Discount Price : ", discountPrice

END METHOD

END CLASS

MAIN

CREATE Electronics e

e.price  $\leftarrow$  50000

CREATE Clothing c

c.price  $\leftarrow$  2000

CREATE Grocery g

g.price  $\leftarrow$  500

CALL e.applyDiscount()

CALL c.applyDiscount()

CALL g.applyDiscount()

STOP

---

**Q9 9. University Staff Hierarchy [BL4 – Analyze]**

10 marks

Analyze the design of a Staff base class extended by TeachingStaff and NonTeachingStaff. Further extend TeachingStaff into Professor and Lecturer (multilevel inheritance). For each level:

- Identify which attributes should be private vs protected
- Override calculateAllowance() at each level
- Analyze how protected access enables inheritance without full exposure. Justify the use of multilevel inheritance and explain how access modifiers enforce data hiding.

OOP Concepts: Multilevel Inheritance | Access Modifiers | Data Hiding

**Your Answer**

```
START
CLASS Staff

    PRIVATE
        staffId
        name

    PROTECTED
        basicSalary

    METHOD setDetails(id, n, salary)
        staffId ← id
        name ← n
        basicSalary ← salary
    END METHOD

    METHOD calculateAllowance()
        DISPLAY "General Staff Allowance"
    END METHOD
END CLASS

CLASS TeachingStaff EXTENDS Staff

    PROTECTED
        subject

    METHOD calculateAllowance()
        allowance ← basicSalary × 20 / 100
        DISPLAY "Teaching Staff Allowance : ", allowance

    END METHOD
END CLASS
```

```
CLASS Lecturer EXTENDS TeachingStaff
```

```
    PRIVATE
```

```
        experience
```

```
    METHOD calculateAllowance()
```

```
        allowance  $\leftarrow$  basicSalary  $\times$  25 / 100
```

```
        DISPLAY "Lecturer Allowance : ", allowance
```

```
    END METHOD
```

```
END CLASS
```

```
CLASS NonTeachingStaff EXTENDS Staff
```

```
    PROTECTED
```

```
        department
```

```
    METHOD calculateAllowance()
```

```
        allowance  $\leftarrow$  basicSalary  $\times$  10 / 100
```

```
        DISPLAY "Non Teaching Staff Allowance : ", allowance
```

```
    END METHOD
```

```
END CLASS
```

```
MAIN
```

```
    CREATE Professor p
```

```
    CALL p.calculateAllowance()
```

```
    CREATE Lecturer l
```

```
    CALL l.calculateAllowance()
```

```
    CREATE NonTeachingStaff n
```

```
    CALL n.calculateAllowance()
```

```
STOP
```

**Q10 10. Smart Home Device Controller [BL4 – Analyze]**

10 marks

Analyze and design a SmartDevice superclass with subclasses: SmartLight, SmartThermostat, and SmartLock. Override an executeCommand(cmd) method in each, where the same command (e.g., 'on', 'off', 'status') produces device-specific behavior. Analyze:

- How polymorphism allows a single controller loop to manage all devices
- Security implications of data hiding in SmartLock (PIN must never be exposed) Design the class hierarchy and explain the OOP principles applied at each level.

OOP Concepts: Polymorphism | Encapsulation | Inheritance

**Your Answer**

```
START
CLASS SmartDevice

    PROTECTED
        deviceId
        deviceName

    METHOD executeCommand(cmd)
        DISPLAY "Executing Command"
    END METHOD
END CLASS

CLASS SmartLight EXTENDS SmartDevice

    PRIVATE
        brightness

    METHOD executeCommand(cmd)
        IF cmd = "on" THEN
            DISPLAY "Smart Light is ON"
        ELSE IF cmd = "off" THEN
            DISPLAY "Smart Light is OFF"
```



```
ELSE IF cmd = "status" THEN
    DISPLAY "Light Status Displayed"
END IF
```

```
END METHOD
END CLASS
```

```
CLASS SmartThermostat EXTENDS SmartDevice
```

```
PRIVATE
    temperature
```

```
METHOD executeCommand(cmd)

    IF cmd = "on" THEN
        DISPLAY "Thermostat Started"
    ELSE IF cmd = "off" THEN
        DISPLAY "Thermostat Stopped"
    ELSE IF cmd = "status" THEN
        DISPLAY "Current Temperature Displayed"
    END IF
```

```
END METHOD
END CLASS
```

```
CLASS SmartLock EXTENDS SmartDevice
```

```
PRIVATE
    PIN
```

```
METHOD executeCommand(cmd)
    IF cmd = "on" THEN
        DISPLAY "Door Locked"
    ELSE IF cmd = "off" THEN
        DISPLAY "Door Unlocked"
    ELSE IF cmd = "status" THEN
        DISPLAY "Lock Status Displayed"
    END IF
END METHOD
END CLASS
```

```
MAIN
CREATE SmartLight light
CREATE SmartThermostat thermostat
```

CREATE SmartLock lock

CREATE list of devices

ADD light, thermostat, lock

FOR each device IN devices DO

    CALL device.executeCommand("status")

    CALL device.executeCommand("on")

END FOR

STOP